

Q1) BFS $\rightarrow$ is a graph traversal algorithm that explores vertices of a graph layer by layer. Starting from a specified source vertex and visiting all its neighbours before moving to the next layer.

BFS using adjacency Matrix

An adjacency matrix is a 2D array $adj[n][n]$ where n is the no. of vertices. If there is edge between u and v , then

$adj[u][v] = 1$ otherwise $adj[u][v] = 0$.

Algorithm:-

Input: n (no. of vertices), adj (adjacency matrix), s (source vertex)

Initialize: A Queue Q to keep track of vertices to visit.

A boolean visited array $visited[n]$ initialized to false.

Start-traversal: Mark the source vertex s as visited and enqueue it

Process the Queue:

- While Q is not empty.

- 1) Dequeue a vertex u from Q

- 2) Process u (e.g. print it or store it in a result list)

- 3) For each vertex v (from 0 to $n-1$)

if $adj[u][v] = 1$ & $visited[v] = false$

- mark v as visited

- enqueue v .

Time-complexity: ^{outer loop} $O(Runs \ V \ times)$ (each vertex enqueued once)

^{inner loop}

$\rightarrow$ Check all v neighbours

T.C = $O(V^2)$

BFS using adjacency list:-

An adjacency list represent a graph as an array of lists. Each index corresponds to a vertex, and the list contains the adjacent vertices.

Algorithm:

- 1.) Input: n (no of vertices), adj list (adjacency list), s (source vertex)
2. Initialize: Queue - Q
boolean visited array visited[]
3. Start Traversal: Mark the source vertex s as visited and enqueue it
4. Process the Queue.
 - while Queue Q is not empty.
 - 1) Dequeue a vertex u from Q
 - 2) Process u .
 - 3) For each vertex v in adj list [u]
 - if visited [u] = false
 - mark v as visited
 - Enqueue v
5. Output - BFS traversal order.

Time Complexity:-

Outer loop:- Runs V times (each vertex is enqueued once)

Inner loop:- iterates over all edges, since each edge is checked once.

$$\text{Overall} = O(V+E)$$

★ DFS using adjacency matrix and adjacency list

→ Algorithm for Iterative

DFS

↓
similar to BFS only difference is
queue is replaced by stack.

1. Create a stack of nodes & visited array.
2. Insert the root in the stack
3. Run a loop till the stack is not empty.
4. Pop the element from the stack and print the element
5. For every adjacent and unvisited node of current node, mark the node and insert it in the stack

matrix

visited every node
even though there
is no connection

$O(V^2)$

list

visited only those
nodes where there
are connections only.

$O(V+E)$

Approach

→ DFS is an algorithm
for traversing or searching
tree or graph data structures.

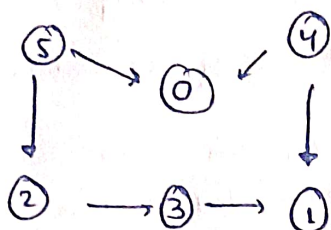
→ Algorithm starts at root
node and explores as far as
possible along each branch
before backtracking.

→ Continue this loop until
no unmarked node is
left.

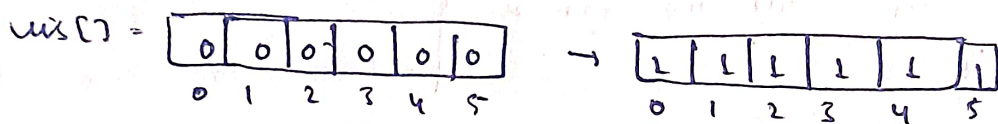
BFS using adjacency list:-

Q.2) Topological sorting:-
for Acyclic directed graph

Linear ordering of vertices such that if there is an edge between u & v , u appears before v in that ordering.

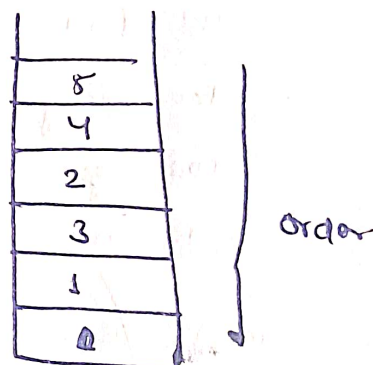


(5 4 2 3 1 0) → one of topological sorting order



- 0 → { }
- 1 → { }
- 2 → { 3 }
- 3 → { 1 }
- 4 → { 0, 1 }
- 5 → { 0, 2 }

Algo- for (i=0 to 5)
{
 dfs(i)
}



dfs(0) → Put 0 on stack mark 0 as visited
↓ No other calls

dfs(1) → Put 1 on stack, mark 1 as visited
↓ No other calls

dfs(2) → dfs(3) while going back, put on stack
↓ No other call

dfs(3) → already visited.
dfs(1)

dfs(4) → Put on stack
↓ dfs(0) dfs(1)
↓ visited.

dfs(5) → Put on stack

Algorithm:-

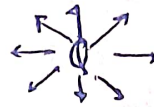
1. Create a stack to store the topological order.
2. Maintain a visited array to keep track of visited vertices.
3. Define a recursive function $dfs(vertex)$ that:-
 - mark current vertex as visited.
 - recursively call itself for all unvisited neighbours of the vertex.
 - Push the vertex onto the stack after exploring all its neighbours.
4. Iterate through all vertices of the graph.
 - If a vertex is unvisited, call the $dfs(vertex)$ function.
5. After processing all vertices, the stack will contain the vertices in topological order (in reverse order of insertion).
6. Pop all elements from the stack to get the topological sort.

Applications

1. Task scheduling - task with dependencies, A must be completed before B.
 - a. e.g. - CPM (critical path method)
 - Program evaluation and review (PERT) technique
2. Dependency resolution in Package manager
3. Deadlock detection in operating system

Q.3) Backtracking → is a systematic algorithmic technique used to solve problems that require finding all or some solutions by building a solution incrementally. If a partial solution does not lead to a valid or desired outcome, the algorithm backtracks (i.e. undoes the last step) and explores alternative possibilities.

N-Queen - Problem



← attacks

In this we put queens on a $N \times N$ chessboard such that

- 1) Each row have one queen
- 2) Each column have one queen
- 3) No Queen should attack each other.

two queens
↓
NO same

→ Placing Conflicts can be check by

• Row condition is automatically handled by algorithm structure.

• Left diagonal Conflicts ($r-c$) → should not be same as of Queen with whom conflict is being checked.

• Right Diagonal Conflicts ($r+c$) ←

Algorithm:-

1. Initialize the board:- Create an $N \times N$ chessboard and set all positions to 0. A value of 1 in a cell indicates a queen is placed there.
2. Start with first row:- Attempt to place a queen in one of the columns of the first row.
3. Check placement validity:- Before placing a queen in a cell (row, col) check if:
 - There is no other queen in the same column.
 - There is no other queen on the left diagonal.
 - There is no other queen on the right diagonal.
4. Recursive backtracking:-
 - If a valid position is found in the current row, place the queen there and move to the next row recursively.
 - If no valid position is found in the current row, backtrack to the previous row, remove the queen from the previously placed position, and try the next column.
5. Terminate when all queens are placed.

If queens are placed successfully in all N rows, print the solution or store it.
6. Backtrack if solution is not found

If you cannot place a queen in a row and all possibilities are exhausted, backtrack to explore other possibilities.
7. Repeat until all solutions are found (if finding multiple solutions)

function solveQueens(board, row, N):

if row == N:

Print Solution (board)

return true

foundSolution = False

for col in range(0, N):

if isSafe(board, row, col, N):

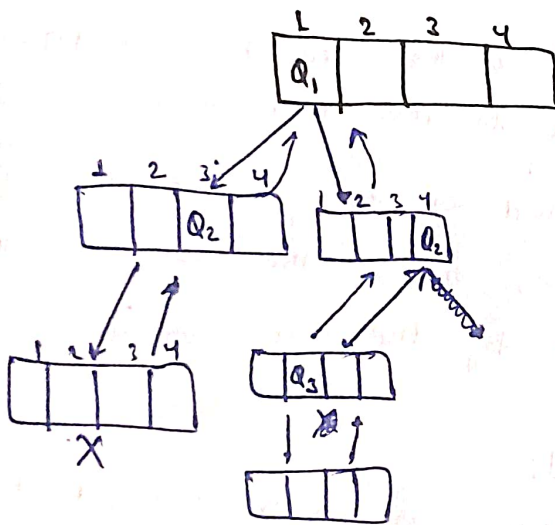
board[row][col] = 1 # Place Queen

foundSolution = solveQueens(board, row+1, N)

board[row][col] = 0 # Backtrack (remove Queen)

return foundSolution:

Solving 4 - Queen Problem

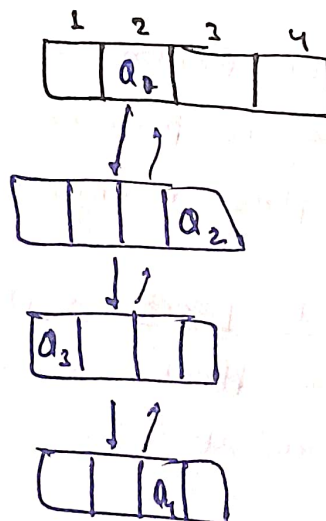


$Q_1 = (1, 2)$

$Q_2 = (2, 4)$

$Q_3 = (3, 1)$

$Q_4 = (4, 3)$



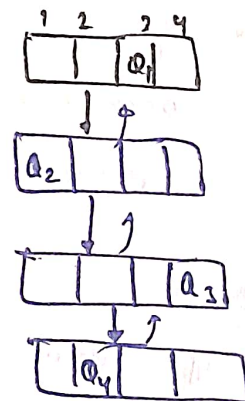
row == N

save result

back track
for other
possible soln.

	1	2	3	4
1	Q ₁	Q ₁		
2				Q ₂
3	Q ₃			
4			Q ₄	

(4x4)



row == N

stop

save
soln.